Can this Python release pipeline publish unattended?

Soup DevOps take-home | 2026-11-10 | Repository: https://github.com/mathalama/soup-devops

Evidence before confidence

The dependency-free CLI prints a greeting from bundled package data. The naive workflow tests a helper from the source checkout on Ubuntu/Python 3.12 and builds distributions. It never installs the published interface. Four separate runner-only mutations all pass that workflow, while installed-artifact verification rejects each. Failed runs remain public.

- Python 3.10: importing tomllib in the CLI requires Python 3.11. Naive GREEN; hardened RED: ModuleNotFoundError on 3.10.

- Windows: importing Unix-only termios breaks startup. Naive GREEN; hardened RED: ModuleNotFoundError on Windows.

- Packaging: excluding greeting.txt leaves source tests intact. Naive GREEN; hardened RED: FileNotFoundError from installed wheel.

- Entry point: metadata points to missing_main. Naive GREEN; hardened RED: ImportError from installed console script.

The fixed package passed run 42808827101: build/lint, Docker and nine clean installs covering Ubuntu, Windows and macOS with Python 3.10/3.12/3.14. The latest run 42810942601 passed build, Docker and all 15 clean install lanes across those operating systems and Python 3.10-3.14. Publish and verify-index were skipped on the `main` branch run. No experimental dispatch can publish.

Release path and silent failures

Build wheel from sdist once; install each distribution into a fresh temporary venv outside the checkout; execute the installed command, check data, metadata and pip consistency. The wheel environment contains no development tools. Sdist build tools use the hash-locked dependency export and build isolation is disabled to prevent fresh unpinned resolution. Publication downloads the tested current-run files and never rebuilds them.

Controls close five failure paths: all Python tool dependencies have exact versions and required SHA256 hashes; Actions and the Docker base use immutable references; release verification restores no dependency cache; current-run commit-named artifacts and version checks reject stale or mismatched files; fork PRs receive read-only permissions, with no publishing credential. Only the tag-push publish job gets OIDC permission, through environment testpypi. These controls address reproducibility, artifact identity and privilege isolation, not just more test coverage.

Operational judgement and GPU design

GPU CI: a design, not a claimed execution

This CLI has no GPU behavior. For a future GPU backend, rent one ephemeral Runpod RTX 4090 only after CPU gates pass. Per release, allow 15 minutes including provisioning for GPU detection, a known-answer calculation against a CPU reference, bounded numerical error, and confirmation that CPU fallback did not occur. Nightly, allow 30 minutes for larger fixtures, memory pressure, repeated results and warmed-up performance checks. Record device/driver/CUDA versions, image digest and release wheel hash.

Assume 30 nightlies at 0.5 hours and four releases at 0.25 hours: 16 GPU hours/month. Runpod's pricing page, checked November 10, lists RTX 4090 at $0.89/hour. Compute is $14.24; 25% contingency makes $17.80. Optional 20 GB standard network storage adds $1.40: $19.20/month before taxes, transfer and other charges. Recheck the deployment quote; these are planning assumptions. No paid GPU was rented.

Run only trusted commits, one GPU job at a time. Reject deterministic retries; permit at most one infrastructure retry within budget. Use controller cleanup plus an independent 45-minute lease reaper. Terminate disposable Pods, since stopped Pods retain billable storage. Enforce a controller-side $25 monthly allowance, in addition to billing alerts. Keep provider credentials in the controller and publishing credentials off GPU runners. Full design: docs/GPU-DESIGN.md.

Verdict and remaining risk

Unattended real PyPI publication is not justified yet. The GitHub `testpypi` environment has a tag-only `v*` deployment policy, but no tagged publication or post-publication installation is evidenced; trusted-publisher registration must also be confirmed. OIDC proves workflow identity, not correctness. Before enabling real PyPI, require proven end-to-end TestPyPI publication and installation, protected release tags/workflow changes, and review of the account and trusted-publisher configuration. The release path should fail closed if any required job fails.

The suite does not prove behavior on every terminal encoding, architecture, Linux distribution, future Python, offline installation or installer version. Hashes cannot detect malicious code already approved in the lockfile. Compromised runners, maintainers or identity providers remain trust assumptions. Lockfile/export parity enforcement, dependency vulnerability checks and signed provenance verification by consumers remain improvements. These limits belong in the release decision rather than being hidden behind green jobs.

What the experiments exposed; AI use

The sharpest result was that an unchanged source test and a successful build coexisted with a console script that could not import its target. Testing from outside the checkout changed the evidence more than adding helper assertions. The remaining concern is confusing reproducible delivery with trustworthy code. Codex proposed and generated the implementation and documents, then executed the listed checks and inspected failure logs. Accepted suggestions include artifact-first verification and isolated OIDC publication; the sdist path was revised after noticing unpinned build resolution, and Python versions were checked explicitly after uv selected 3.14 locally. The candidate must review this account and add their own observations before submission; no independent human verification is claimed.

References: PyPI publishing, GitHub secure use, Runpod lifecycle. Execution details and limitations: tasks/evidence.md.